SYSTEM OWNER'S HANDBOOK

End-to-End Workflows (Part 6)

Multi-tenant clinic management platform

Status as of 2026-10-28 · code is the source of truth · written for a software-engineer owner

How to read this document

This is part of the **System Owner's Handbook** for a multi-tenant clinic management product, called "the Platform" throughout. It is written for the product owner, a software engineer, and every chapter can be read on its own.

Status labels. Every feature carries exactly one label, based on the code as of 2026-10-28:

- **[Implemented]**: built end to end.
- **[Partial]**: some of it exists; the text says exactly what is missing.
- **[Not implemented]**: planned, seeded, disabled or only mentioned. It is never described as working.
- **[Unverified]**: the code was read but the behaviour could not be confirmed; the text says what is unconfirmed.

Source of truth. The code is the source of truth. Where older documentation disagreed with the code, the handbook follows the code. Nothing was executed while writing: behaviour is described from reading the code.

Figures. Every figure has a number, a caption and a paragraph that describes it in words. Figures captioned "screenshot pending" are placeholders; the paragraph after them describes the screen from the code. The few real screenshots were taken on 2026-10-23 and show a former working name of the product.

Names. "Organization" is one clinic business (the tenant); "location" is one of its branches. File paths and code names appear in a short "Technical references" line at the end of each feature. Example people and places are fictional.

Table of contents

Part 6 — End-to-end workflows

Chapter 46 — Workflow: Onboarding a New Clinic 4
Chapter 47 — Workflow: Onboarding a Staff Member 9
Chapter 48 — Workflow: The Patient Journey 13
Chapter 49 — Workflow: A Dental Lab Case 17
Chapter 50 — Workflow: Closing the Clinic Day 21
Chapter 51 — Workflow: Month-End Payroll 24
Chapter 52 — Workflow: Correcting an Invoice and Refunding Money 27
Chapter 53 — Workflow: Subscription Expiry and Recovery 31
Chapter 54 — Workflow: How an Access Change Reaches Every Screen 35
Chapter 55 — Workflow: From Purchase Order to Consumption 38

Chapter 46 — Workflow: Onboarding a New Clinic

This chapter follows one new clinic from nothing to a clinic that is ready to receive patients. There are two ways in: the owner signs up alone, or a super admin creates the organization by hand. It is written for the product owner, who needs to know which steps are automatic, which need a super admin, and where the two paths differ. Each step names the module, its status and the chapter that explains it in full.

46.1 Who is involved

- **Owner**: the person who runs the clinic business. On the self-signup path this person creates the clinic. On the other path an owner login is created for them.
- **Super admin**: the Platform operator. Creates organizations by hand, entitles features, records payments and can act inside any clinic.
- **Example Clinic**: the fictional organization used in this chapter, with a Main location.
- **Sample staff**: employees added at the end (Chapter 47 covers staff onboarding).

46.2 The steps

Figure 46.1 shows both paths and the shared steps that follow them.

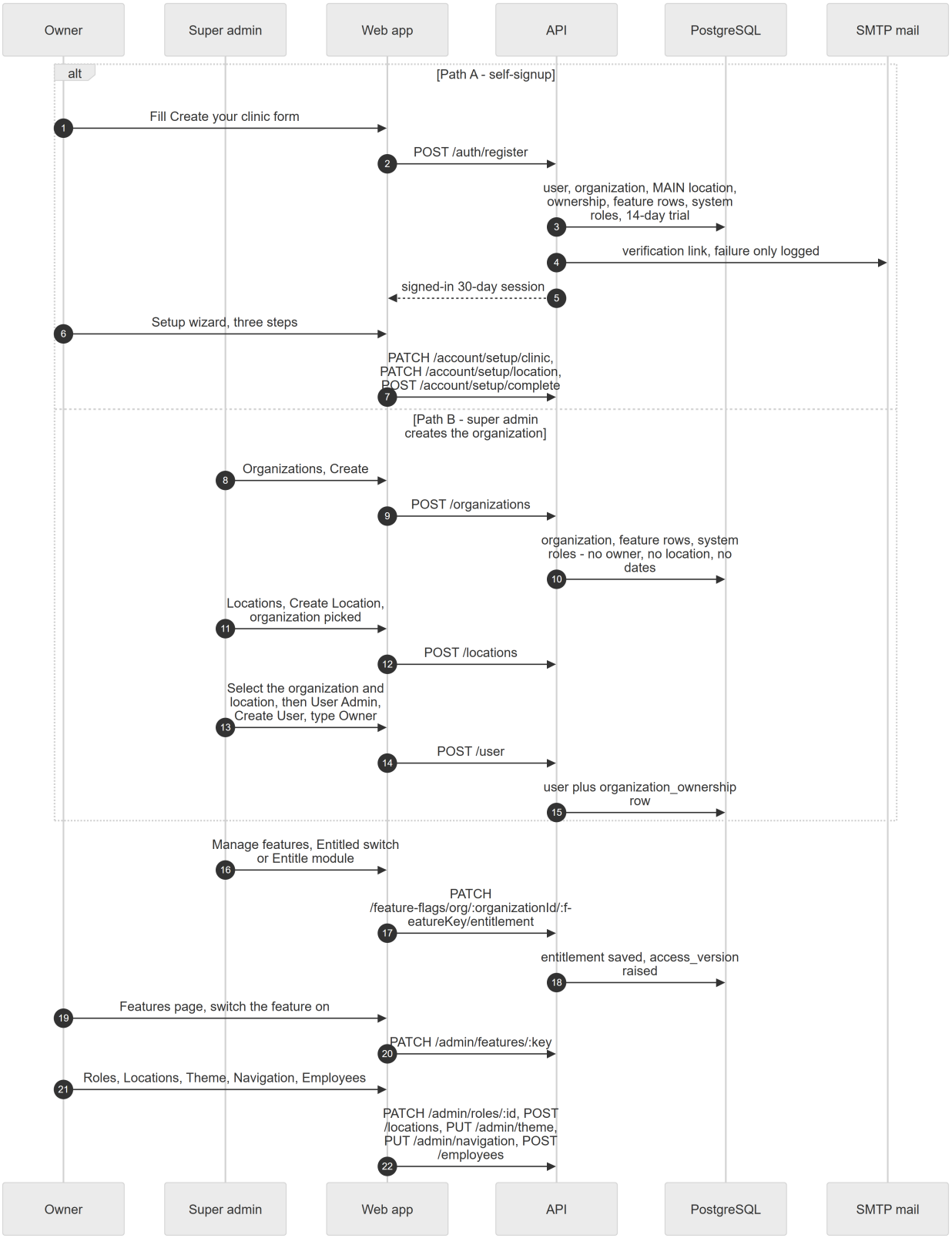


Figure 46.1 — Two ways to create a clinic, then the shared set-up steps.

The diagram has six participants from left to right: Owner, Super admin, Web app, API, PostgreSQL and SMTP mail. It has 22 numbered messages. A box labelled "alt" holds the two paths. In Path A (messages 1 to 7) the Owner fills the "Create your clinic" form, the Web app posts to `POST /auth/register`, and the API writes the user, the organization, the MAIN location, the ownership row, the feature rows, the system roles and a 14-day trial to PostgreSQL. The API sends a verification link through SMTP mail (a failure is only logged) and returns a signed-in 30-day session. The Owner then goes through the three-step setup wizard, and the Web app saves it with two PATCH calls and one POST that completes setup. In Path B (messages 8 to 15) the Super admin creates the

organization from the Organizations screen; the API stores the organization, its feature rows and system roles but no owner, no location and no dates. The Super admin then creates a location, selects the organization and location, and creates a user of type Owner in User Admin, which writes the user and an ownership row. After the box, both paths continue: the Super admin entitles features (messages 16 to 18), the Owner switches features on (19 and 20), and the Owner sets up roles, locations, theme, navigation and employees (21 and 22).

Step 1a — Self-signup (Path A)

Module: Subscriptions and Onboarding. **Status:** [Implemented]. The visitor opens `/register`, enters clinic name, full name, email, an optional phone and a password. The Platform creates the owner login, the organization, one location with code `MAIN`, the ownership link, a 14-day trial on the Professional plan, the feature rows and the five system roles, then signs the owner in. Chapter 45 describes this in full: a visitor registers, and a trial starts at once.

Step 1b — Verify the email address

Module: Subscriptions and Onboarding. **Status:** [Partial]. The owner receives a link that is valid for 48 hours and opens `/verify-email`. Verification is informational: nothing is blocked while the address is unverified, and a failed email send is only logged. Chapter 45 and the Authentication chapter (Chapter 56) cover it.

Step 1c — Setup wizard

Module: Subscriptions and Onboarding. **Status:** [Implemented]. An owner whose setup is pending is sent to `/setup` from any other page. Step "Clinic" collects name, clinic types, phone, email, timezone and currency. Step "Location & tax" collects the main location's details and the sales tax percentage. Step "Your team" only explains that staff are added later; its finish button clears the pending flag without checking that the first two steps were saved. Saving the clinic step copies timezone and currency to every location of the clinic (Chapter 45).

Step 2 — Super admin creates the organization (Path B)

Module: Organizations and locations. **Status:** [Partial]. `POST /organizations` works, and it seeds the feature rows and the five system roles. The Organizations screen's form offers plan, clinic type, timezone and currency values that the API rejects, so a form filled with those fields fails with HTTP 400 (found by reading both files, not by running the app). The result is an unmanaged clinic: no owner, no location, no trial and no plan limits (Chapter 9 and Chapter 45).

Step 3 — Give the clinic a location and an owner (Path B)

Module: Organizations and locations, and User Admin. **Status:** [Partial]. A super admin adds a location from the Locations screen by choosing the organization. There is no field on the organization screen to name an owner. The code creates an owner login through User Admin with the account type "Owner", which uses the active organization from the request header. A super admin therefore has to select that organization and a location before creating the owner. This sequence is deduced from the code and was not run. Chapter 8 explains ownership; Chapter 56 explains User Admin.

Step 4 — Entitle features

Module: Features. **Status:** [Implemented]. A super admin opens "Manage features" for the clinic. Every feature has an **Entitled** switch, an optional trial end date and an **Enabled** switch; "Entitle module" acts on a whole module. The seeding at clinic creation sets entitled and enabled equal to each feature's default. Several features that clinics rely on are off by default and need this step: among them `billing.invoices`, `billing.commissions`, `inventory.core`, `dental.lab_orders`, `reports.financial` and `reports.clinical`. Without `billing.invoices`, completing a consultation creates no charges (Chapter 31). Chapter 10 covers the three switches.

Step 5 — Owner enables features

Module: Features. **Status:** [Implemented]. The owner opens **Features** (`/admin/features`) and switches on the features in the plan. A locked feature, which is not entitled or whose trial ended, cannot be switched on. Turning a feature off shows an impact list of the permissions and roles affected, and a location can switch a feature off for itself only (Chapter 10).

Step 6 — Roles

Module: Roles and permissions. **Status:** [Implemented]. The five system roles (Admin, Doctor, Receptionist, Assistant, Accountant) already exist. The owner may clone one or build a custom role at **Roles & permissions** (`/admin/roles`) with None, Basic or Full per module. System roles are read-only (Chapter 11).

Step 7 — Locations

Module: Organizations and locations. **Status:** [Implemented]. The owner adds further branches from Locations. The plan's location limit applies (Professional allows 3) and an expired subscription refuses the write. The edit screen for locations sends fields the API refuses, so changing a location afterwards may fail [Partial] (Chapter 9).

Step 8 — Theme and navigation

Module: Customization. **Status:** [Implemented]. At `/admin/theme` the owner sets the display name (default "Clinic Portal"), logos, favicon, brand colour (default a blue), sidebar style and default light, dark or system mode. A location may override the name and logos. At `/admin/navigation` the owner arranges the sidebar for the whole clinic. Both need their own permission (`admin.theme.manage` , `admin.nav.manage`). Changing the sidebar layout raises the access version; the theme save does not appear to (Chapter 58 covers customization; Chapter 54 describes how access changes spread).

Step 9 — Staff

Module: Employees. **Status:** [Implemented]. The owner or an Admin adds employees with roles and logins. Chapter 47 follows that workflow.

46.3 What can go wrong

- **Duplicate email at sign-up.** User creation rejects it. If clinic creation fails after the user is created, the new user is deleted so no orphan account remains.
- **Rate limit.** Registration is limited to 5 per hour, verification to 10 per 15 minutes and resend to 3 per hour.
- **Feature set after signup is smaller than expected.** Features that are off by default must be entitled by a super admin. Clinics that have not been entitled for billing see no automatic charges.
- **Seeding failure.** After the clinic is committed, seeding the feature rows and roles can fail; the failure is only logged, so a clinic can exist with no roles (Chapter 45).
- **Plan limit.** Adding a fourth location on Professional gives `PLAN_LIMIT` .
- **Super admin path cannot be finished in one screen.** Owner creation needs an organization context, which needs a location first.
- **Expired trial.** After the trial and 3 grace days the clinic is read-only; setup and account paths stay writable (Chapter 53).

46.4 Data left behind

- `users` : the owner login, with the email verification columns.
- `organizations` : plan, `trial_ends_at` , `onboarding_pending` , `access_version` .
- `organization_ownership` : one primary owner on Path A; one non-primary owner on Path B.
- `locations` and `location_settings` : at least one location with the tax rate.
- `org_feature_flags` (entitled and enabled per feature) and `employment_roles` (the five system roles).
- `config_values` : theme settings. `nav_layouts` : the sidebar layout, if the owner saved one.
- `audit_logs` : feature entitlement changes and nav layout changes.
- Employees and assignments after Step 9.

Chapter 47 — Workflow: Onboarding a Staff Member

This chapter follows one new staff member, Dr. Example One, from the first employee record to the first sign-in. It is written for the product owner, who needs to know how a person, a role, a login, working hours and pay terms fit together, and what the Platform cannot do. The workflow touches Employees, Roles, User Admin, Authentication and Commissions.

47.1 Who is involved

- **Admin** (or the owner): holds `employees.manage`, `admin.users.manage` and `compensation.manage`. Only the Admin system role holds them by default.
- **Dr. Example One**: the new employee. Becomes a **staff login** once a user account is linked.
- **Example Clinic**, Main location.

47.2 The steps

Figure 47.1 shows the whole flow.

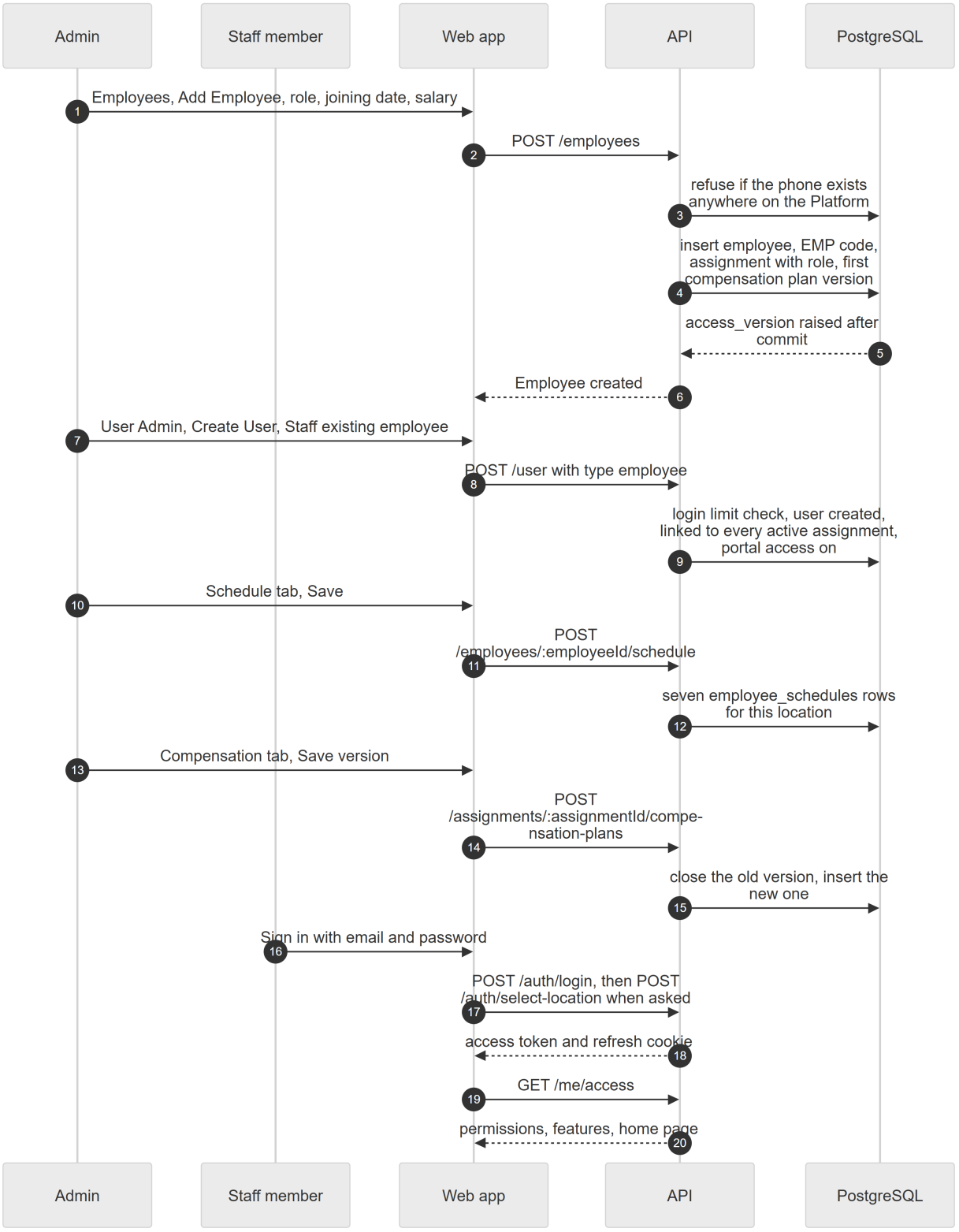


Figure 47.1 — From the Add Employee form to the first sign-in.

The diagram has five participants: Admin, Staff member, Web app, API and PostgreSQL, and 20 messages. The Admin fills the Add Employee form and the Web app calls `POST /employees`. The API first refuses the request if the phone number exists anywhere on the Platform, then inserts the employee, an `EMP-` code, the assignment with its role and the first compensation plan version. PostgreSQL raises the organization's access version after the commit, and the API answers "Employee created". Next the Admin opens User Admin and creates a staff login (`POST /user` with type employee); the API checks the login limit, creates the user and links it to every active assignment with portal access on. The Admin saves the weekly schedule (`POST /employees/:employeeId/schedule`, seven rows for this location) and, if needed, a new compensation plan version

(`POST /assignments/:assignmentId/compensation-plans`). Finally the Staff member signs in, the Web app calls `POST /auth/login` and, when asked, `POST /auth/select-location`, and then `GET /me/access`, which returns permissions, features and the home page.

Step 1 — Employee record, assignment and role

Module: Employees. **Status:** [Implemented]. On **Employees**, **Add Employee** asks for Full Name, Phone, Designation, Role, Employment Type and Joining Date. The server rejects a phone that already exists on the Platform (the employee record is shared across clinics). It generates the employee code (`EMP-001` style, counting the organization's assignments), inserts the assignment at the active location with the chosen role, and adds the first compensation plan version starting on the joining date: salaried with the entered salary, unless a commission plan was chosen (Chapter 37, Chapter 35). The role decides what the person may do at that location; a login alone grants nothing.

Step 2 — Staff login

Module: Employees and User Admin. **Status:** [Implemented]. There are two routes. The **Portal Access** switch on the employee form creates a login with an email and a password of at least 6 characters, linked only to the active location's assignment. **User Admin** → **Create User** → **Staff (existing employee)** needs a password of at least 8 characters, checks the plan's login limit and links the login to all the person's active assignments in the organization. Only the second route checks the limit (Chapter 37). The user receives no verification email when created by an administrator (Chapter 56).

Step 3 — Weekly schedule

Module: Employees. **Status:** [Implemented]. On the **Schedule** tab the Admin switches days on or off, sets start and end times and a slot length of 15, 30, 45 or 60 minutes, and saves seven rows. Booking checks these hours: a doctor with no working block that day cannot be booked (Chapter 29). The Save button is visible to people without `employees.manage`, but the server refuses them. Availability overrides and approved leave are stored but do not affect booking [Partial] (Chapter 37).

Step 4 — Compensation plan

Module: Commissions. **Status:** [Implemented]. On the **Compensation** tab the Admin picks Salaried, Commission or Both and an effective date. A new version closes the old one on the previous day. Commission is allowed only for roles that can conduct consultations, and the feature `billing.commissions` must be on. A second change on the same day replaces the first only if no charge used it (Chapter 35).

Step 5 — First sign-in

Module: Authentication and Access control. **Status:** [Implemented]. The staff member signs in with the email and password. If they have one location, it becomes active at once; otherwise the app shows the location picker. `GET /me/access` returns what they can do and the role's home page, for example `/queue/doctor` for a Doctor (Chapter 8, Chapter 11, Chapter 56).

What is not possible: a second location

Module: Employees. **Status:** [Not implemented]. There is no endpoint that assigns an existing employee to another location or organization. `CreateAssignmentDto` exists but nothing uses it. Add Employee at a second location fails because the phone number is already taken. A doctor who works at two locations therefore cannot be set up through the Platform (Chapter 37).

47.3 What can go wrong

- **Duplicate phone** gives 409 "An employee with this phone number already exists". A duplicate CNIC is caught only by a database index and returns a generic 500 error.
- **Duplicate login email** gives 409.
- **Commission plan refused**: 403 `COMMISSIONS_DISABLED` when the feature is off, 422 `COMMISSION_ROLE_CANNOT_CONSULT` for a non-consulting role.
- **Login limit** reached gives `PLAN_LIMIT` on the User Admin route.
- **Wrong or missing role**: the person signs in but sees a restricted sidebar or the no-access page.
- **Read-only mode**: every step is refused with 402.
- The person is saved before the login and the assignment, outside the final transaction, so a failure part-way can leave a person record with no assignment.

47.4 Data left behind

- `employees` (person), `employee_org_location_assignments` (role, location, `user_id`, `has_portal_access`).
- `users` (login), `employee_schedules` (seven rows per location).
- `compensation_plans` and `commission_rules`.
- `organizations.access_version` raised by the assignment save.

Chapter 48 — Workflow: The Patient Journey

This chapter follows Sample Patient A through one visit at Example Clinic, from registration to a printed receipt. It is written for the product owner. It follows the order the code really uses, which differs from a naive reading in two places: the invoice is created when payment is collected, and a lab case is a separate flow that is billed separately.

48.1 Who is involved

- **Receptionist R1**: registers, books, checks in and collects payment.
- **Dr. Example One**: calls the patient, consults, prescribes, completes the visit.
- **Sample Patient A**: the patient.
- **Accountant** and **Admin**: may also collect payments and correct invoices.

48.2 The steps

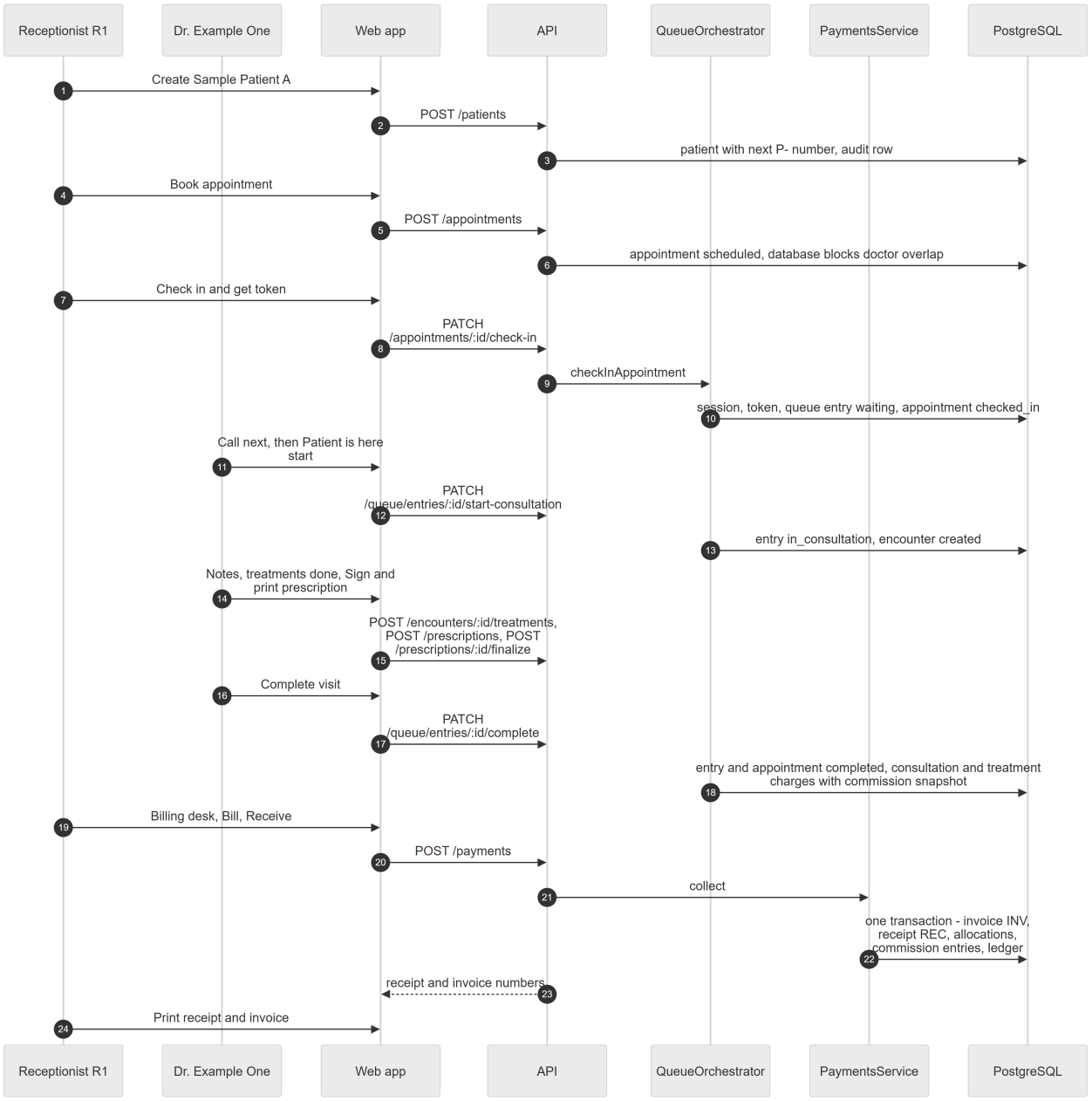


Figure 48.1 — One visit from registration to receipt.

The diagram has seven participants: Receptionist R1, Dr. Example One, Web app, API, QueueOrchestrator, PaymentsService and PostgreSQL, and 24 messages. R1 creates the patient (`POST /patients`) and the API stores the patient with the next `P-` number and an audit row. R1 books an appointment (`POST /appointments`); it is saved as scheduled and the database blocks a doctor overlap. On arrival R1 clicks "Check in and get token" (`PATCH /appointments/:id/check-in`), the API hands over to QueueOrchestrator, which writes the session, the token, a queue entry in state waiting and sets the appointment to checked in. Dr. Example One calls the next patient and presses "Patient is here, start" (`PATCH /queue/entries/:id/start-consultation`); the entry becomes in consultation and the encounter is created. The doctor records notes and treatments, signs the prescription (`POST /encounters/:id/treatments` , `POST /prescriptions` , `POST /prescriptions/:id/finalize`) and presses "Complete visit" (`PATCH /queue/entries/:id/complete`); in one transaction the entry and appointment become completed and the consultation and treatment charges are created with the commission snapshot. R1 then opens the billing desk, presses Bill and Receive (`POST /payments`); PaymentsService runs one transaction that issues the invoice, records the receipt, allocates the money, writes commission entries and the ledger row. The API returns the receipt and invoice numbers and R1 prints them.

Step 1 — Registration

Module: Patients. **Status:** [Implemented]. **Create** on `/patients` takes Full Name and Phone. The phone must be unique inside the organization. The server assigns `P-` plus a five-digit number and writes an audit row. The check-before-save endpoint exists, but no screen calls it [Partial] (Chapter 28).

Step 2 — Appointment

Module: Appointments. **Status:** [Implemented]. The booking checks the doctor works at that location, that the slot fits a working block, that the doctor has no overlapping booking and the daily limit. A booking has no token yet. A walk-in skips this step (Chapter 29).

Step 3 — Check-in

Module: Appointments and Queue. **Status:** [Implemented]. Check-in is allowed only on the booking's own day and up to the location's maximum lateness (45 minutes by default). It opens the doctor's queue session if needed and issues a token such as A-007. A patient who is too late is offered "Convert to walk-in" (Chapter 29, Chapter 30).

Step 4 — Queue

Module: Queue. **Status:** [Implemented]. The doctor starts the queue, then **Call next** picks the next patient by the smart or first-come rules. Reception and the TV display update live (Chapter 30).

Step 5 — Consultation

Module: Consultation. **Status:** [Implemented]. Starting the consultation creates the encounter, the clinical record of this visit, in the same transaction. The workspace shows the patient summary and allergy strip, and auto-saves notes (Chapter 31).

Step 6 — Treatment

Module: Consultation. **Status:** [Implemented]. The doctor taps procedure chips from the service catalog. Each becomes a treatment. The doctor never sees prices. A treatment typed by name has no price and is later priced by reception (Chapter 31).

Step 7 — Prescription

Module: Prescriptions. **Status:** [Implemented]. In the workspace panel the doctor adds medicines and presses "Sign & print". The prescription is optional and has no effect on billing (Chapter 32).

Step 8 — Lab case (separate flow)

Module: Dental Lab. **Status:** [Not implemented] from the consultation workspace. The workspace has no lab-order button, and a lab order has no link to an encounter in the data model (Chapter 31). A lab order is actually created from the sidebar: **Lab orders** → **New lab order**, where the user picks the patient, the lab, the ordering doctor and the items, then presses **Submit to lab**. The API accepts an optional appointment and treatment on the order, but the order form in the web app does not appear to send them [Unverified]. Chapter 49 follows the lab case. Lab work is billed by its own invoice, not by the clinic's charges.

Step 9 — Automatic charges

Module: Billing. **Status:** [Implemented] with a condition. When the doctor completes the visit, the Platform creates one consultation charge and one charge per treatment, priced from the service catalog with any doctor-specific price. This runs only when the feature `billing.invoices` is enabled; that feature is off by default (Chapter 31, Chapter 34). A charge with no price is marked "needs pricing" and blocks invoicing until someone sets a price. Each charge stores a snapshot of the doctor's commission terms (Chapter 35).

Step 10 — Payment and invoice

Module: Billing. **Status:** [Implemented]. On the **Billing desk**, "ready to bill" lists patients with unbilled charges. **Collect payment** previews the split and then saves. The invoice is issued at this moment, taking the next `INV` number, with the location's sales tax added on top. Surplus money becomes the patient's advance. A patient who leaves without paying can still be invoiced with **Issue invoice** from the patient's Billing tab (Chapter 34).

Step 11 — Receipt

Module: Billing. **Status:** [Implemented]. The payment takes the next `REC` number. The result screen prints the receipt and the invoice in A4 or 80 mm thermal layout. There is no server-generated PDF receipt [Not implemented] (Chapter 34).

48.3 What can go wrong

- Duplicate phone on registration gives 409.
- Booking conflicts: `DOCTOR_DOUBLE_BOOKED`, `DAILY_LIMIT_REACHED`, outside working hours.
- Check-in too late (`LATE_LIMIT_EXCEEDED`), wrong day (`NOT_TODAY`), queue closed (`SESSION_NOT_OPEN`).
- A stale queue screen gives 409 `VERSION_CONFLICT`.
- With `billing.invoices` off, completion succeeds and no charge appears.
- A charge needing a price blocks the invoice ("Set a price for ...").
- Payment errors: missing reference for an account that requires one, allocation above balance.
- In read-only mode every write gives 402.

48.4 Data left behind

`patients`, `audit_logs`, `appointments`, `queue_sessions`, `queue_entries`, `queue_events`, `outbox`, `idempotency_keys`, `patient_encounters`, `treatments`, `prescriptions` and children, `charges`, `billing_audit_logs`, `invoices`, `payments`, `payment_allocations`, `document_sequences`, `ledger_entries`, `commission_entries`.

Chapter 49 — Workflow: A Dental Lab Case

This chapter follows one lab case, a zirconia crown for Sample Patient A, from the order to the delivered work, and then follows the money: what the patient is billed, what the clinic owes the lab and what the doctor earns. It is written for the product owner. The case involves three modules (Dental Lab, Lab Finance, Commissions) and two sides: the clinic and the lab, which is not a tenant of the Platform. The lab feature `dental.lab_orders` is off by default; a super admin entitles it and the owner switches it on.

49.1 Who is involved

- **Dr. Example One**: creates and submits the order, decides try-ins, approves and delivers.
- **Receptionist R1**: receives shipments at the front desk.
- **Accountant**: bills the patient for lab work, approves vendor bills, pays the lab.
- **Example Dental Lab**: the **lab**. Its people are **lab members** (lab admin, lab technician) who use the **lab portal**, or a contact who acts through a **magic link**.
- **Admin**: may act on behalf of a lab that does not answer.

49.2 The steps

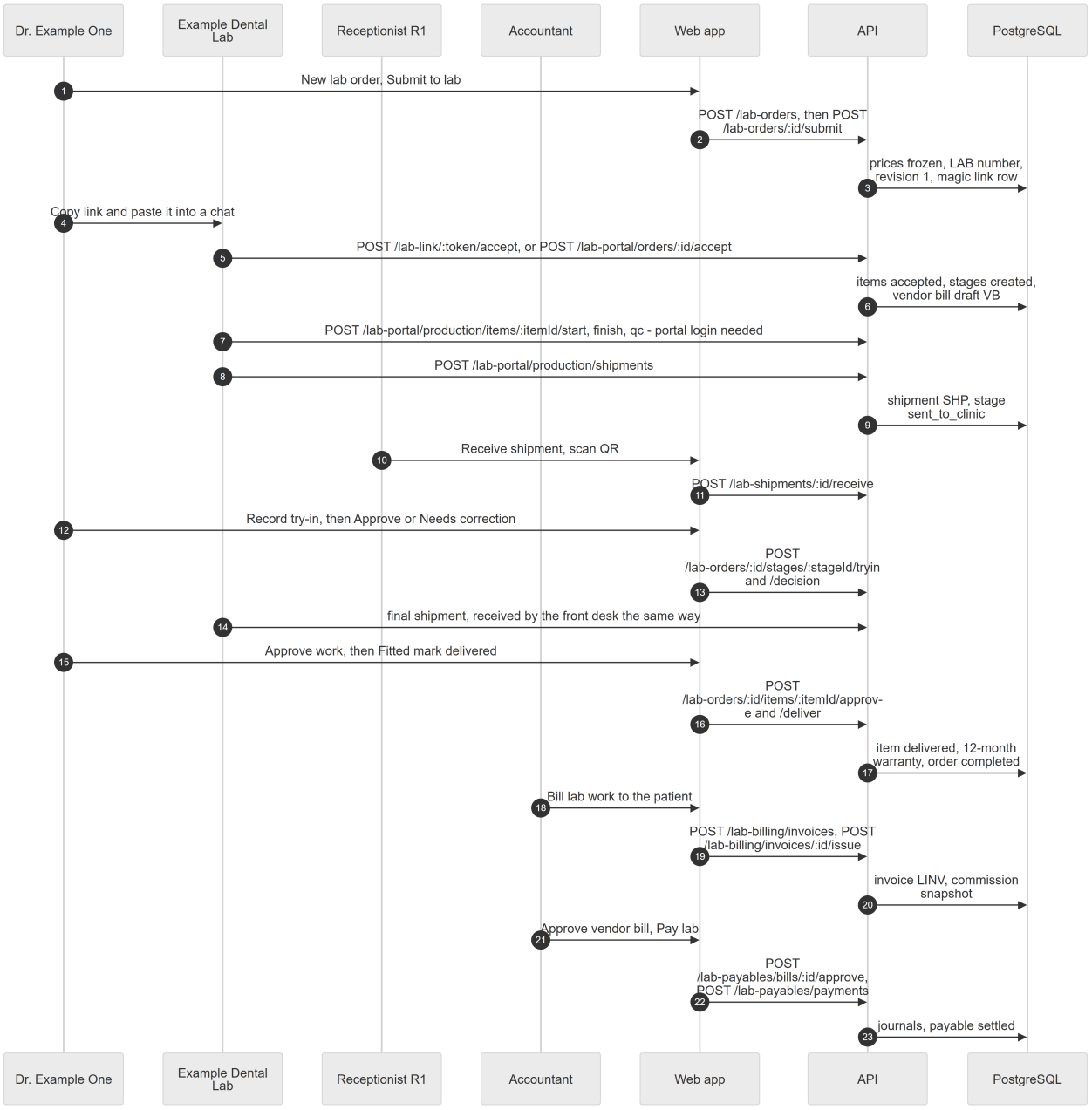


Figure 49.1 — Order to delivery, then the money.

The diagram has seven participants: Dr. Example One, Example Dental Lab, Receptionist R1, Accountant, Web app, API and PostgreSQL, and 23 messages. The doctor submits the order (`POST /lab-orders` , then `POST /lab-orders/:id/submit`); the API freezes the prices, assigns a LAB number, stores revision 1 and a magic link row. The doctor copies the link and pastes it into a chat. The lab accepts, through the link (`POST /lab-link/:token/accept`) or the portal (`POST /lab-portal/orders/:id/accept`), and the API marks the items accepted, creates the stages and drafts a vendor bill. The lab then works through the portal: start, finish and QC (`POST /lab-portal/production/items/:itemId/start` and the following actions) and dispatches a shipment (`POST /lab-portal/production/shipments`). The front desk receives it by scanning the QR code (`POST /lab-shipments/:id/receive`). The doctor records the try-in and approves or asks for correction (`.../stages/:stageId/tryin` and `/decision`). After the final shipment is received, the doctor approves and marks the work delivered (`.../items/:itemId/approve` and `/deliver`), which starts a 12-month warranty and completes the order. The Accountant bills the patient (`POST /lab-billing/invoices` and `/issue`), approves the vendor bill and pays the lab (`POST /lab-payables/bills/:id/approve` , `POST /lab-payables/payments`).

Step 1 — Create and submit the order

Module: Dental Lab. **Status:** [Implemented]. **Lab orders** → **New lab order**: choose patient, lab, ordering doctor, priority and date needed; add items (service, options, quantity, teeth, form answers); **Submit to lab**. The lab must be an active **lab link** of the clinic. At submit the Platform checks the forms, freezes the lab and patient prices of every line, assigns a number such as LAB-0001 and queues a message for the lab members. Every line needs a lab price and a mapped clinic treatment on the submit day (Chapter 40).

Step 2 — Reach the lab

Module: Dental Lab. **Status:** [Partial]. Three routes exist. The lab portal gives members the order in their own inbox [Implemented] (Chapter 41 covers the portal). A **magic link** opens one order without a login and allows view, accept, reject and ask a question only. **Send link to lab** queues a message, but the only registered provider for WhatsApp, SMS and email writes the message to the server log, so nothing reaches the lab; **Copy link** returns the link once and the doctor pastes it into a chat by hand (Chapter 40, Chapter 44). A coordinator holding both `lab.orders.create` and `lab.links.manage` can record the lab's answer on its behalf [Implemented].

Step 3 — Lab accepts

Module: Dental Lab and Lab Finance. **Status:** [Implemented]. Accept needs an expected date. Items move to accepted, the stage rows are copied from the item's stage template, and a draft **vendor bill** is created or updated at the frozen lab cost (Chapter 42).

Step 4 — Production stages

Module: Lab Portal. **Status:** [Implemented]. Production actions (start, assign a technician, finish, QC, dispatch a shipment) are lab portal endpoints for members. A lab that has only a magic link and no portal login cannot perform them, and the clinic's proxy actions cover only accept, reject, clarification and expected date. The clinic follows progress on the order's Production panel (Chapter 40).

Step 5 — Try-in

Module: Dental Lab. **Status:** [Implemented]. For items with try-in stages the doctor presses **Record try-in** after the patient visit, then **Approve** or **Needs correction** with notes. A correction closes the round and opens the same stage as the next round. No appointment is created or reminded automatically; the event for a due try-in is never emitted (Chapter 40).

Step 6 — Shipment and receive

Module: Dental Lab. **Status:** [Implemented]. The front desk opens the incoming shipment or scans its QR code, sets condition and quantity per line and presses **Receive**. A parcel that never arrives can be marked lost with a reason.

Step 7 — Approve and deliver

Module: Dental Lab. **Status:** [Implemented]. The doctor presses **Approve work**, then **Fitted, mark delivered**, which starts a 12-month warranty. Rejecting received work opens a remake request that the lab may dispute. When all items are approved or delivered the order is completed.

Step 8 — Money: patient lab invoice

Module: Lab Finance. **Status:** [Implemented]. This is not the clinic's normal invoice: lab work is billed through its own tables and numbers (`LINV`). The Accountant opens the submitted order and presses **Bill N item(s)**, which starts an **estimate**; **Issue invoice** gives it a number and due date. The lines copy the patient prices frozen at submit. This can be done any time after the order is submitted, not only after delivery. Payments, advances and credit notes follow (Chapter 42).

Step 9 — Money: vendor bill and payment to the lab

Module: Lab Finance. **Status:** [Partial]. The draft vendor bill appears under **Lab payables** → **Bills**. The Accountant presses **Approve**, setting the lab's invoice number, due date and tax, which books the debt. **Pay lab** records a payment and splits it over approved bills. Editing the bill header and adding manual lines exist only in the API (Chapter 42).

Step 10 — Money: lab-line commission

Module: Commissions. **Status:** [Implemented] (backend only by design). When the lab invoice is issued, each line stores the doctor's plan and rate. As the patient pays, the doctor earns commission on the money collected minus the lab's cost, so the doctor is paid on the clinic's margin. Reversals and credit notes give it back. It needs `billing.commissions` on and feeds My earnings and payroll like any other commission (Chapter 35).

49.3 What can go wrong

- Submit fails when a line has no lab price, no mapped treatment, a per-tooth service without teeth or answers that fail the form (`LAB_ORDER_INVALID` , `LAB_PRICE_MISSING`).
- The lab link is not active: `LAB_NOT_LINKED` .
- A magic link dies after 24 hours by default, on revoke, when the order closes or when the lab is suspended.
- A lab that never answers: use the proxy actions.
- Parcels may arrive damaged, short or not at all.
- A rejected final piece needs a remake; who pays is decided in the remake request.
- Double billing is blocked: an item can be on only one live lab invoice.
- A bill of 0 cannot be approved; approval is refused if the draft changed since it was read.
- Four seeded notification events are never emitted, including overdue orders.

49.4 Data left behind

`lab_orders` , `lab_order_items` , `order_revisions` , `magic_links` , `order_item_stages` , `shipments` , `item_warranties` , `status_history` , `audit_logs` , `notification_outbox` , `vendor_bills` , `vendor_bill_lines` , `vendor_payments` and allocations, `lab_invoices` , `lab_invoice_lines` , `commission_entries` with a lab invoice line, and journal rows in `ledger_journals` .

Chapter 50 — Workflow: Closing the Clinic Day

This chapter describes how a clinic day ends at Example Clinic: the doctor closes the queue, late bookings become no-shows, reception reviews the day's collection, the accountant closes the counter, and the Platform's overnight jobs run. It is written for the product owner. Some of the work is done by people and some by background jobs in the worker process (Appendix E lists all jobs). Job times use the server clock, not each clinic's timezone.

50.1 Who is involved

- **Dr. Example One**: closes the queue.
- **Receptionist R1**: reads the day's collection.
- **Accountant**: closes the counter. Admin may also do this.
- **Worker process**: the one process that runs scheduled jobs.

50.2 The steps

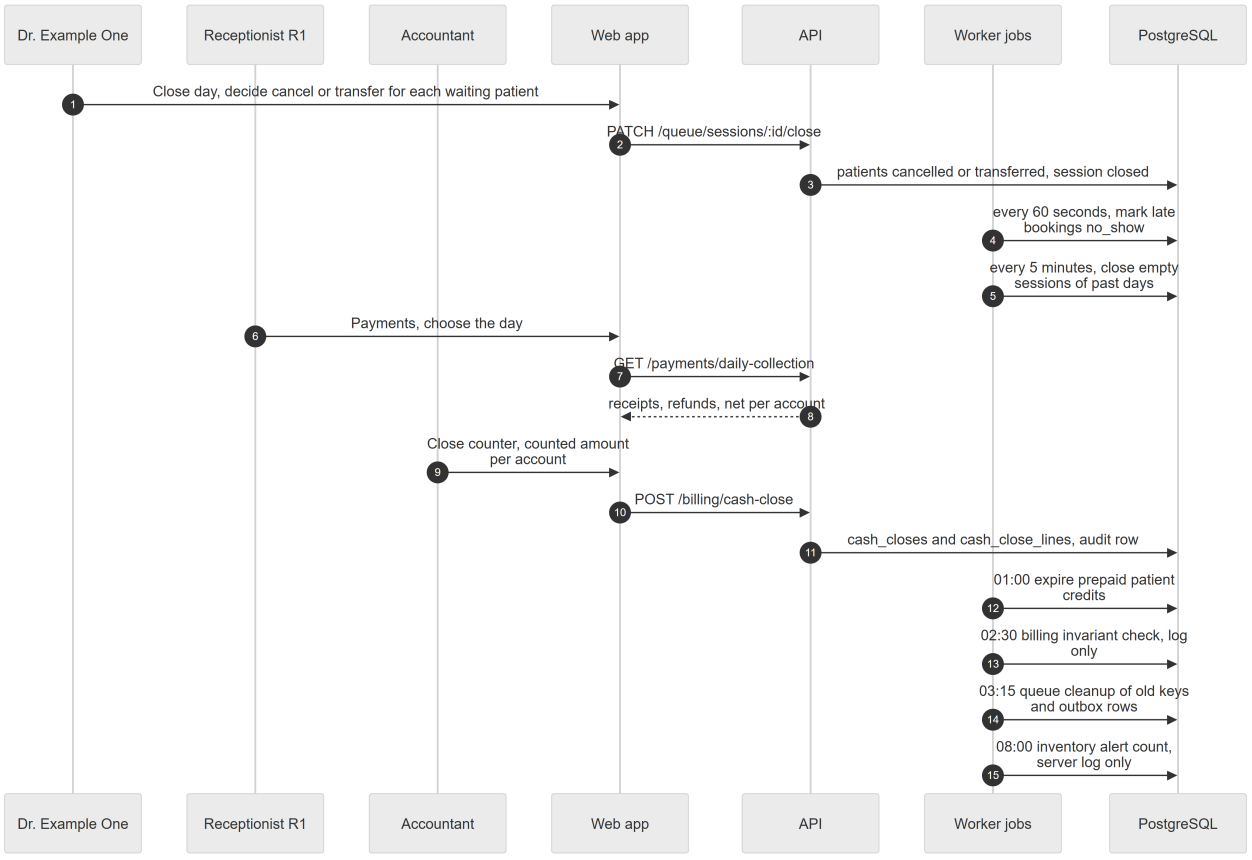


Figure 50.1 — People first, then the jobs that run on their own.

The diagram has seven participants: Dr. Example One, Receptionist R1, Accountant, Web app, API, Worker jobs and PostgreSQL, and 15 messages. The doctor closes the day and decides cancel or transfer for each waiting patient (`PATCH /queue/sessions/:id/close`); the API cancels or transfers them and closes the session. In parallel, the worker marks late bookings as no-shows every 60 seconds and closes empty sessions of past days every 5 minutes. Receptionist R1 opens Payments and picks the day (`GET /payments/daily-collection`) and the API returns receipts, refunds and net per account. The Accountant closes the counter with a counted amount per

account (`POST /billing/cash-close`) and the API stores the close and its lines with an audit row. The worker then runs four overnight jobs: expire prepaid credits at 01:00, billing invariant check at 02:30, queue cleanup at 03:15 and inventory alert count at 08:00.

Step 1 — Queue day-end

Module: Queue. **Status:** [Implemented]. In **My Queue**, **Close day** is disabled while a patient is with the doctor. The "Close today's queue" dialog lists every remaining patient (waiting, skipped or at a test) with **Cancel** as the default or "Transfer to Dr. ..." for waiting patients only. Confirming cancels or moves them and closes the session. Cancelling a booked patient closes the appointment as cancelled. Everything succeeds or nothing does (Chapter 30).

Step 2 — Automatic day-end

Module: Queue. **Status:** [Implemented]. Every 5 minutes the worker closes sessions of a past service day if they are empty. A session that still has patients is only logged as an alert and is never cancelled automatically, so someone must decide. The "past day" is judged by each location's timezone and day cut-off, not by the server date.

Step 3 — No-show marking

Module: Appointments and Queue. **Status:** [Implemented]. Staff can press **No show** on a scheduled or confirmed booking (`appointments.cancel`). Otherwise the worker does it every 60 seconds once start time plus maximum lateness (45 minutes by default) has passed. The job writes with plain SQL, so there is no queue event, no audit row and no realtime message. A no-show no longer shows Check in in the web app, so a late patient usually has to be added as an ordinary walk-in (Chapter 29).

Step 4 — Daily collection

Module: Billing. **Status:** [Implemented]. On **Payments**, the day picker (future days not allowed) shows receipts by their payment date and refunds by their refund date per payment account, with the net in hand (Chapter 34).

Step 5 — Cash close

Module: Billing. **Status:** [Implemented]. **Close counter** computes for each payment account the expected money since the previous close, asks for the counted amount on accounts marked "Count physically at cash close", and requires a note when there is a difference. A close is a permanent record with a printable slip and a red "Flagged" badge when the difference exceeds the alert amount. Closing never blocks taking payments, and a close cannot be edited or deleted. It needs `billing.cash_close` (Admin, Accountant).

Step 6 — Overnight jobs

Module: Several. Times are server time (Appendix E).

Time	Job	Status
01:00	Patient credit expiry: marks credits past their expiry date as expired	[Implemented] (credit packages have no screen, so little to expire in practice)
02:30	Billing invariant check: re-checks ten money rules and logs any break	[Implemented], log only
03:15	Queue cleanup: deletes idempotency keys older than 24 hours and published outbox rows older than 7 days	[Implemented]

Time	Job	Status
08:00	Inventory alerts: counts low and expiring items per location	[Partial], server log only

The table lists the four daily jobs with what each does and its status. The credit job reads the server date rather than the location's, so a credit can expire a few hours early or late for a clinic in another timezone. Nobody is notified by the invariant check or the inventory alert; an operator must read the logs (Chapter 34, Chapter 39).

50.3 What can go wrong

- Closing the queue while someone is with the doctor: 409 `DOCTOR_BUSY`.
- A remaining patient without a decision: 422 `INVALID_TRANSITION`.
- A skipped or at-test patient cannot be transferred, only cancelled or brought back.
- Counter close: 422 when a count or a note is missing, or when the close period is stale.
- Overnight times differ from clinic local time when the server is in another timezone.
- Jobs run only in a worker (or all-in-one) process; if no worker runs, none of the jobs run. Exactly one worker must run.
- The invariant check only logs; a violation is not repaired or announced.

50.4 Data left behind

`queue_sessions` (closed), `queue_entries`, `queue_events` and `outbox` (for closes done by users), `appointments` (cancelled or no-show), `cash_closes`, `cash_close_lines`, `billing_audit_logs`, `patient_credits` (expired), and server log lines for the invariant check and the inventory alerts.

Chapter 51 — Workflow: Month-End Payroll

This chapter follows payroll for one month at Example Clinic, from the automatic run on the 1st to the payment record. It is written for the product owner. Payroll combines two things: the base salary from each employee's compensation plan and the doctor commission earned but not yet paid out. The Platform records the payout; it does not move money.

51.1 Who is involved

- **Scheduler** (the worker process): runs the job.
- **Admin**: holds `payroll.manage` (feature `hr.payroll`, on by default). No other system role holds it.
- **Dr. Example One**: an employee with a salary and commission.

51.2 The steps

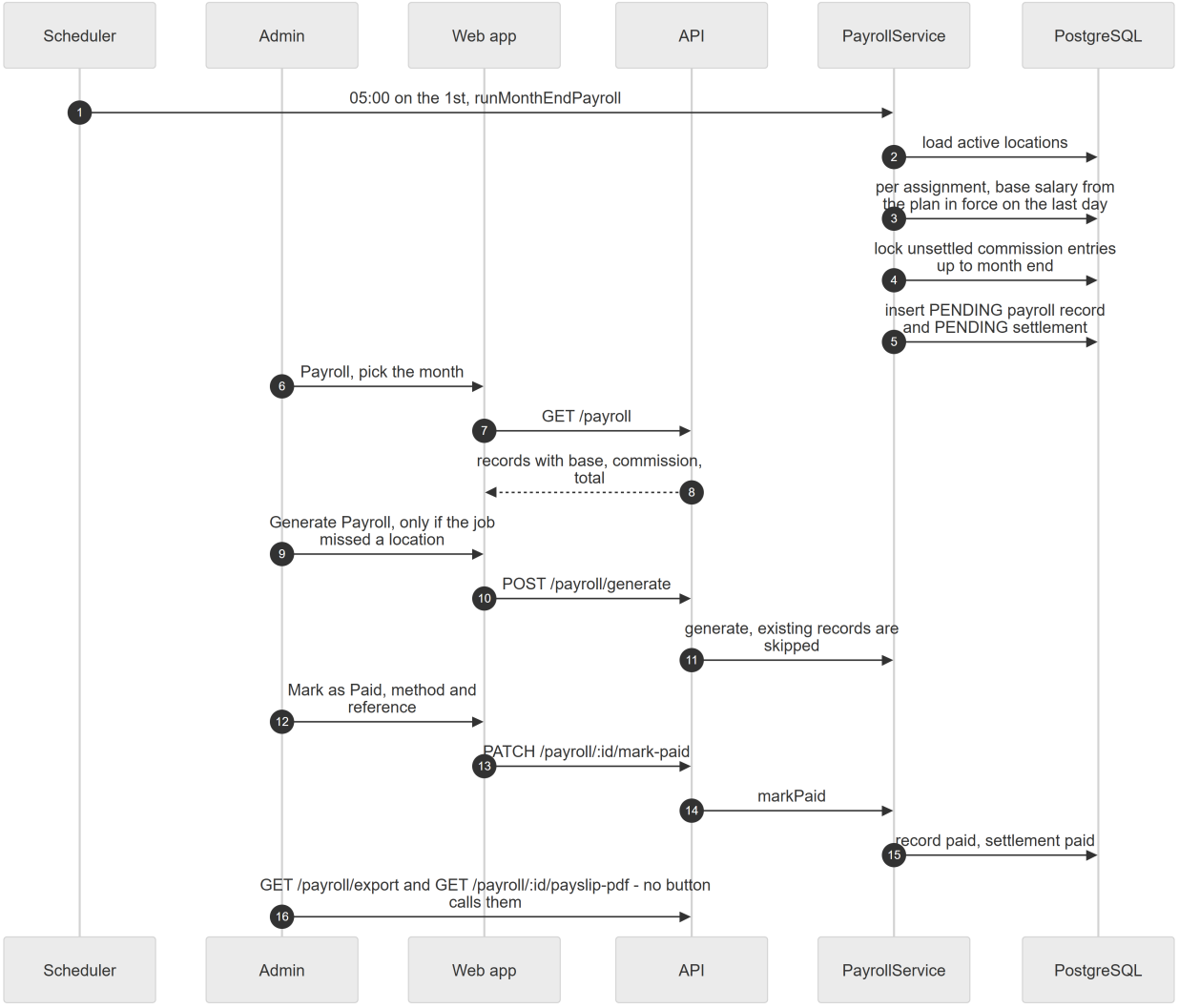


Figure 51.1 — The automatic run, the optional manual run and the payment.

The diagram has six participants: Scheduler, Admin, Web app, API, PayrollService and PostgreSQL, and 16 messages. At 05:00 on the 1st the Scheduler calls the month-end run on PayrollService, which loads the active locations. For each assignment it reads the base salary from the plan in force on the last day of the month, locks

the unsettled commission entries up to month end, and inserts a pending payroll record and a pending settlement. The Admin opens Payroll for the month (`GET /payroll`) and sees records with base, commission and total. If the job missed a location, the Admin presses Generate Payroll (`POST /payroll/generate`) and existing records are skipped. The Admin presses Mark as Paid with a method and reference (`PATCH /payroll/:id/mark-paid`); the service sets the record and the settlement to paid. The last message shows the CSV export and payslip PDF endpoints, which no button calls.

Step 1 — Automatic generation

Module: Payroll. **Status:** [Implemented]. The cron expression is `0 5 1 * *` with no timezone option, so it fires at 05:00 on the 1st in the server's timezone. A code comment says 00:05; the code wins. The month is taken from the server's own date (the previous month), not from each location's timezone. The job covers every active location of every organization, each in its own transaction, so one failing location does not stop the others. It does not check that the organization is active or that `hr.payroll` is on. It runs without a request, so the subscription read-only guard, which acts on web requests, does not apply to it (deduced from the guard being an HTTP guard) (Chapter 38, Appendix E).

Step 2 — Base plus commission

Module: Payroll and Commissions. **Status:** [Implemented]. For each active assignment: skip if a record already exists for that employee, location and month; otherwise base salary = the plan version in force on the month's last day (else the first day); commission = the sum of unsettled commission entries created before the end of the month's last clinic day. A net-negative sum is not settled and waits for a later month. Total = base + commission. An employee with neither is listed as "no salary" and gets no record (Chapter 38, Chapter 35).

Step 3 — Settlement lock

Module: Commissions. **Status:** [Implemented]. The entries are row-locked and tied to a **settlement** so they cannot be paid twice. Entries are never edited; a reversal after settlement lands as a negative entry in a later month's settlement (Chapter 35).

Step 4 — Manual generation

Module: Payroll. **Status:** [Implemented]. **Generate Payroll** on `/payroll` takes a month and year, refuses future months and generates for the whole active location, even when pressed from an employee tab. It is safe to run after the automatic job.

Step 5 — Review and edit

Module: Payroll. **Status:** [Implemented]. Records start as pending. Edit, delete and create-by-hand exist. A record that paid out commission cannot be deleted. Deleting an unpaid record releases its commission back to the pool. Putting a record on hold has an endpoint but no button; the edit dialog's status field can set it [Partial].

Step 6 — Mark paid

Module: Payroll. **Status:** [Implemented]. **Mark as Paid** needs a method (cash, bank transfer or cheque) and optional reference and notes. The record becomes paid with time and payer, and the settlement becomes paid in the same transaction. No money moves and nothing is posted to the general ledger or expenses. Setting status to paid through the edit dialog does not pay the settlement, which is a trap: a later delete would release the commission.

Step 7 — Payslip and export

Module: Payroll. **Status:** [Partial]. `GET /payroll/:id/payslip-pdf` returns an A4 PDF (Latin text only) and `GET /payroll/export?month&year` returns a CSV. Neither has a button in the web app, so they can be called only through the API. Deductions are always 0.00, net equals gross, and the designation column is empty.

51.3 What can go wrong

- Server timezone differs from the clinic's, so "the 1st at 05:00" falls at another local time, and the month boundary can shift near midnight.
- No worker process is running: nothing is generated.
- A location error is logged and skipped; no in-app notification tells anyone.
- Future month: 400 "Cannot generate payroll for future months".
- "Payroll already marked as paid" on a second payment.
- Paying through the edit dialog leaves the commission unpaid.
- No deductions, tax, overtime or unpaid-leave handling exist [Not implemented].
- Payroll changes are not written to the general audit log.

51.4 Data left behind

`payroll_records` (pending, then paid with `paid_at`, method, reference), `commission_settlements` (period, total, payroll record id, status), `commission_entries.settlement_id`. Records made by the job have no `created_by`.

Chapter 52 — Workflow: Correcting an Invoice and Refunding Money

This chapter follows what the Accountant does when an issued invoice turns out to be wrong: preview and save an edit that creates a new revision, cancel an unpaid invoice, refund money, or undo a payment first. It is written for the product owner, who needs to know what stays on record and how doctor commission follows each change. Invoices are never deleted; every change adds to the history.

52.1 Who is involved

- **Accountant** (or Admin): holds `billing.invoice.edit`, `billing.invoice.void` and `billing.refund.issue`. The last two are marked high risk.
- **Sample Patient A**: the patient whose invoice is corrected.
- **Dr. Example One**: the doctor whose commission adjusts.

52.2 The steps

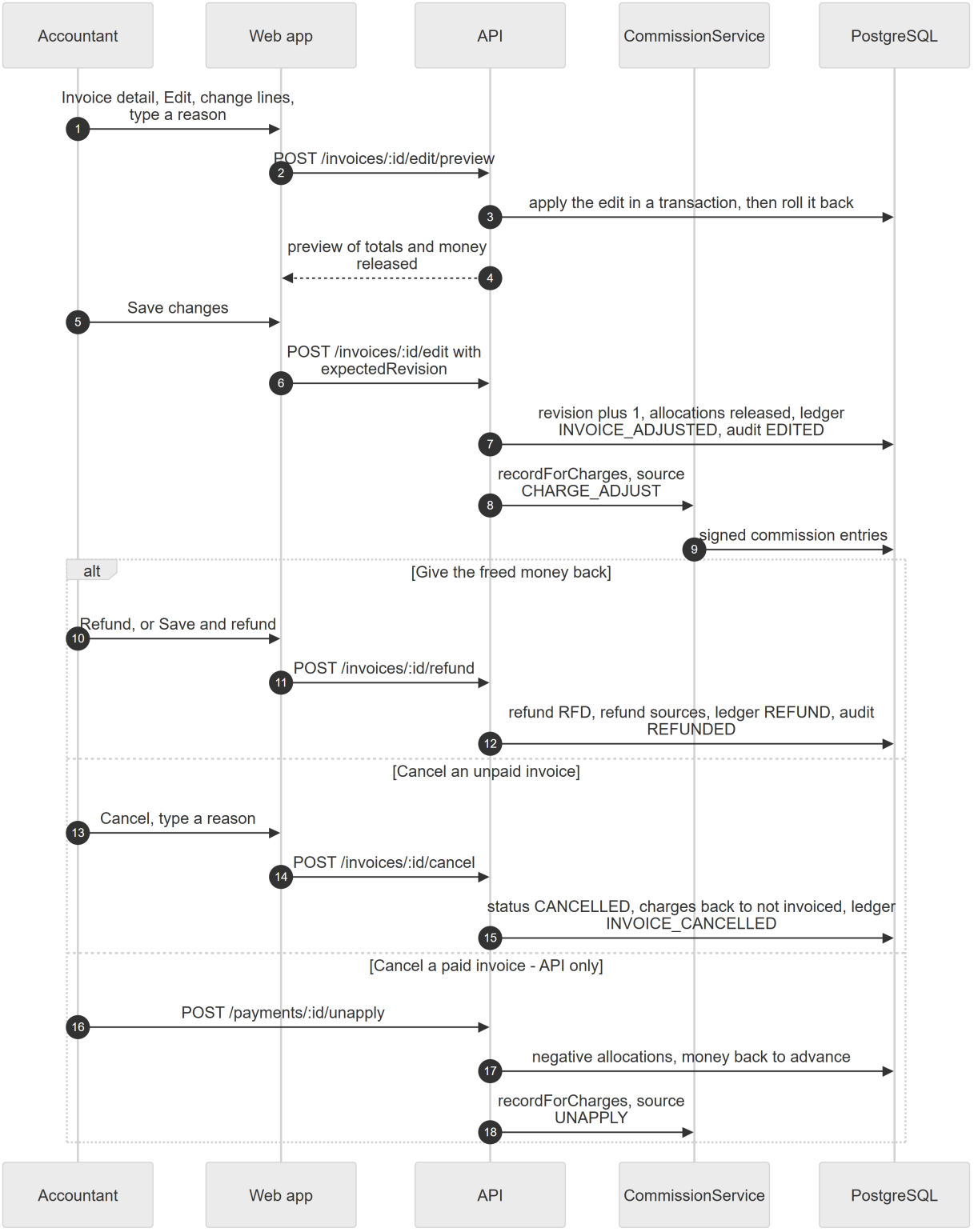


Figure 52.1 — Edit with preview, then refund or cancel.

The diagram has five participants: Accountant, Web app, API, CommissionService and PostgreSQL, and 18 messages. The Accountant opens the invoice, presses Edit, changes lines and types a reason; the Web app calls `POST /invoices/:id/edit/preview`, the API applies the edit inside a transaction and rolls it back, and returns the preview. The Accountant presses Save changes (`POST /invoices/:id/edit` with the expected revision); the API raises the revision, releases allocations, posts an adjustment to the ledger and writes an audit row, then asks CommissionService to record the signed commission entries. An "alt" box has three outcomes. To give freed

money back, the Accountant presses Refund or Save and refund (`POST /invoices/:id/refund`) and the API writes the refund, its sources, a ledger row and an audit row. To cancel an unpaid invoice, the Accountant presses Cancel with a reason (`POST /invoices/:id/cancel`), and the invoice becomes cancelled with its charges back to not invoiced. To cancel a paid invoice, an API client must first unapply the payment (`POST /payments/:id/unapply`), which writes negative allocations and reverses commission.

Step 1 — Preview the edit

Module: Billing. **Status:** [Implemented]. On the invoice detail, **Edit** lets the Accountant change quantities, prices and discounts, void lines, add unbilled charges or add a manual line, with a reason of 5 to 300 characters. The preview really runs the edit in a transaction and discards it, so the numbers match what saving will do (Chapter 34).

Step 2 — Save a revision

Module: Billing. **Status:** [Implemented]. Saving raises the revision number; the invoice number never changes. If a line's payable shrinks, the allocation above the new amount is released, newest first, and moves to the patient's advance. Older revisions can be printed from audit snapshots. If two people edit at once, the second gets `BILLING_INVOICE_REVISION_CONFLICT` . Voiding a line needs `billing.invoice.void` ; changing a catalog price needs `billing.price.override` .

Step 3 — Commission follows

Module: Commissions. **Status:** [Implemented] (backend only). On every edit the Platform recomputes the doctor's commission target per charge and inserts only the difference as a signed entry. A voided charge brings its commission to zero. Entries already settled in payroll are never changed; the correction lands in the next settlement (Chapter 35).

Step 4 — Cancel an unpaid invoice

Module: Billing. **Status:** [Implemented]. **Cancel** needs a reason and is shown only when nothing is paid. The invoice keeps its totals and prints with a CANCELLED watermark, the number stays used, the charges return to Ready to bill, and the patient's statement gets a credit. Cancelled is final.

Step 5 — Unapply a payment (backend only)

Module: Billing. **Status:** [Partial]. `POST /payments/:paymentId/unapply` takes back all or part of a payment's allocations and sends the money to advance. It adds negative rows and deletes nothing, and the receipt stays. No screen calls it, so cancelling an invoice that has payments cannot be done from the web app (`BILLING_INVOICE_HAS_PAYMENTS` : "Unapply its payments before cancelling this invoice").

Step 6 — Refund

Module: Billing. **Status:** [Implemented]. A refund always comes out of the patient's advance. An **advance refund** returns unused money from the patient's Billing tab. An **invoice refund** returns money released from an invoice, optionally in the same step as the edit that released it. The accountant chooses the account under "Paid out from"; it is not forced to be the account that received the payment. Sources are chosen automatically: payments released from that invoice first, latest first, then other advance, newest first. A refund takes the next `RFD` number, needs a reason of 5 to 300 characters, and cannot be undone. Cash close subtracts refunds per account (Chapter 34, Chapter 50).

52.3 What can go wrong

- `BILLING_INVOICE_REVISION_CONFLICT` : someone else changed the invoice; reload.
- `BILLING_INVOICE_CANCELLED` : a cancelled invoice is read-only.
- `BILLING_REFUND_EXCEEDS_ADVANCE` and `BILLING_REFUND_EXCEEDS_INVOICE_REFUNDABLE` : the amount is above what can be returned.
- `BILLING_REFERENCE_REQUIRED` for an account that needs a reference.
- `BILLING_UNAPPLY_EXCEEDS_ALLOCATION` : more than was applied.
- Wrong refunds cannot be reversed; the only remedy is a new payment.
- Read-only mode refuses all of these with 402.

52.4 Data left behind

`invoices` (revision, status), `charges` , `payment_allocations` (including negative and release rows), `payments` , `refunds` , `refund_sources` , `ledger_entries` (INVOICE_ADJUSTED, INVOICE_CANCELLED, REFUND), `billing_audit_logs` (EDITED, CANCELLED, REFUNDED, UNAPPLIED with snapshots), `commission_entries` (signed differences), `document_sequences` .

Chapter 53 — Workflow: Subscription Expiry and Recovery

This chapter follows Example Clinic when its trial runs out and the clinic is brought back by a manual payment. It is written for the product owner. The Platform never charges anyone: the clinic pays outside the Platform and a super admin records the payment, which extends the period. The subscription status is never stored; it is worked out from two dates every time it is read.

53.1 Who is involved

- **Owner**: sees the banners and pays.
- **Other staff**: see warning banners and rejected saves.
- **Super admin**: records the payment and may change plan, dates and limits.

53.2 The steps

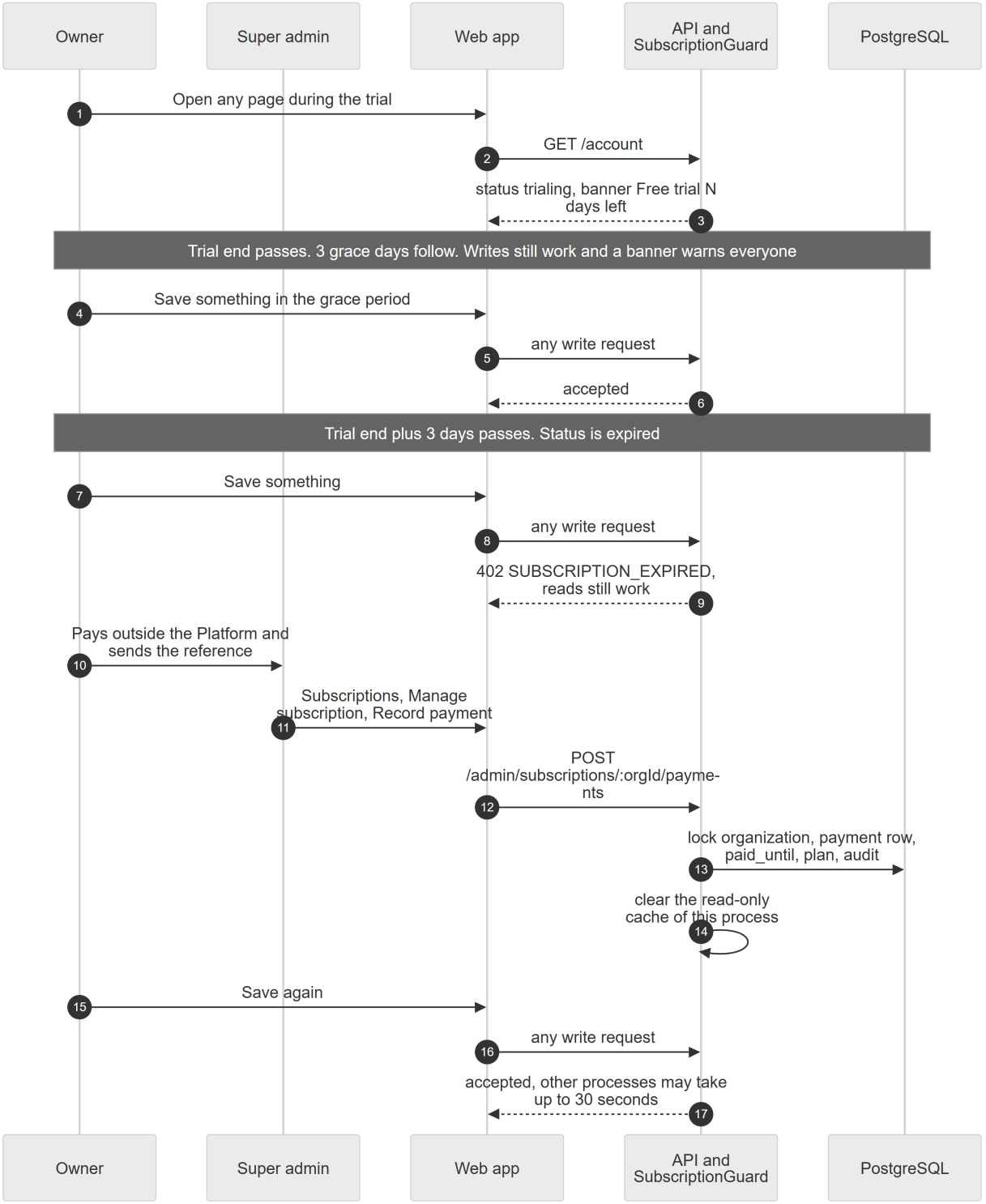


Figure 53.1 — Trial, grace, read-only, payment, back to active.

The diagram has five participants: Owner, Super admin, Web app, API and SubscriptionGuard (drawn as one box), and PostgreSQL, and 17 messages plus two notes. During the trial the Owner opens a page, the Web app calls `GET /account`, and the API answers status trialing with the banner "Free trial, N days left". A first note says that after the trial end passes there are 3 grace days in which writes still work and a banner warns everyone; a save in that period is accepted. A second note says that once the trial end plus 3 days has passed, the status is expired. A save then returns 402 `SUBSCRIPTION_EXPIRED`, while reads still work. The Owner pays outside the Platform and sends the reference to the Super admin, who opens Subscriptions, Manage subscription, Record payment (`POST`

`/admin/subscriptions/:orgId/payments`). The API locks the organization, writes the payment row, sets `paid_until` and the plan and writes an audit entry, then clears the read-only cache of its own process. The Owner saves again and the write is accepted; other server processes may take up to 30 seconds to notice.

Step 1 — Trial

Module: Subscriptions and Onboarding. **Status:** [Implemented]. Self-signup gives 14 days on the Professional plan (3 locations, 15 logins). Owners see "Free trial, N days left" with a **See plans** link. A clinic with no trial date and no paid date is "unmanaged": no limits, never expires (Chapter 45).

Step 2 — Grace period

Module: Subscriptions and Onboarding. **Status:** [Implemented]. The period end is the later of the trial end and the paid-until date. For 3 days after it (`GRACE_DAYS`) the clinic works normally; everyone sees a warning that the clinic turns read-only in N days. A plan close to its end shows owners "Your plan renews in N days".

Step 3 — Read-only mode (402)

Module: Subscriptions and Onboarding. **Status:** [Implemented]. After the grace days, the last global guard refuses every write with HTTP 402 and code `SUBSCRIPTION_EXPIRED`. Reads (GET, HEAD, OPTIONS) pass, public routes pass, super admins pass, and paths starting with `/auth/`, `/account`, `/me/` and `/admin/subscriptions` stay writable. The banner says "Subscription expired. The clinic is read-only" to owners and "read-only until the owner renews the plan" to others. Everything is affected, including the queue and billing. Jobs in the worker are not requests, so they keep running; this is deduced from the guard being an HTTP guard [Unverified].

Step 4 — Pay outside the Platform

Module: Subscriptions and Onboarding. **Status:** [Implemented]. The owner opens **Settings** → **Plan & billing** and reads the "How to pay" text, which the super admin edits on the Subscriptions page (up to 2,000 characters). Methods recorded are bank transfer, JazzCash, EasyPaisa, cash, cheque and other.

Step 5 — Super admin records the payment

Module: Subscriptions and Onboarding. **Status:** [Implemented]. On **Subscriptions**, **Manage subscription** → **Record payment** takes plan, months (1 to 36), amount, method, date, reference and notes. The new period starts where the current trial or paid period ends, or now if that is over, so early payment loses no days. The plan on the clinic becomes the plan paid for, so paying for Starter during a Professional trial lowers the limits at once.

Step 6 — Active again

Module: Subscriptions and Onboarding. **Status:** [Implemented]. With `paid_until` in the future the status is active and writes resume. The decision is cached for 30 seconds per server process, so a renewal can take up to 30 seconds to be seen by a process other than the one that took the payment. A super admin can also set a future trial end or clear both dates to make the clinic unmanaged from the **Plan & limits** tab.

53.3 What can go wrong

- The owner never sees a renewal reminder email; none is sent.
- The grace period is only 3 days.
- Recording a payment overwrites the plan and can lower the limits.

- Staff cannot read the cause of a rejected save unless the banner is visible; the API message is "Your subscription has expired. The clinic is read-only until it is renewed."
- The Organizations screen's plan selector sends values the API rejects, so use the Subscriptions screen for plan changes (Chapter 45).
- Nothing charges the clinic, so a payment the super admin forgets to record keeps the clinic read-only.

53.4 Data left behind

`subscription_payments` (plan, months, amount, method, reference, period start and end, `recorded_by`), `organizations` (`paid_until`, `subscription_plan`, limits), `audit_logs` (before and after values). The status itself is not stored.

Chapter 54 — Workflow: How an Access Change Reaches Every Screen

This chapter follows what happens after an owner edits a role, a feature switch or the sidebar layout: how the change reaches the server's caches and the staff members' open browser tabs without anyone signing out. It is written for the product owner. The key idea is the **access version**, a counter on the organization that goes up by one on every change to who can do what.

54.1 Who is involved

- **Owner** (or an Admin with `admin.roles.manage`): makes the change.
- **Staff member**: has the Platform open in a browser tab.
- **API and worker processes**: keep a cache of each user's access.

54.2 The steps

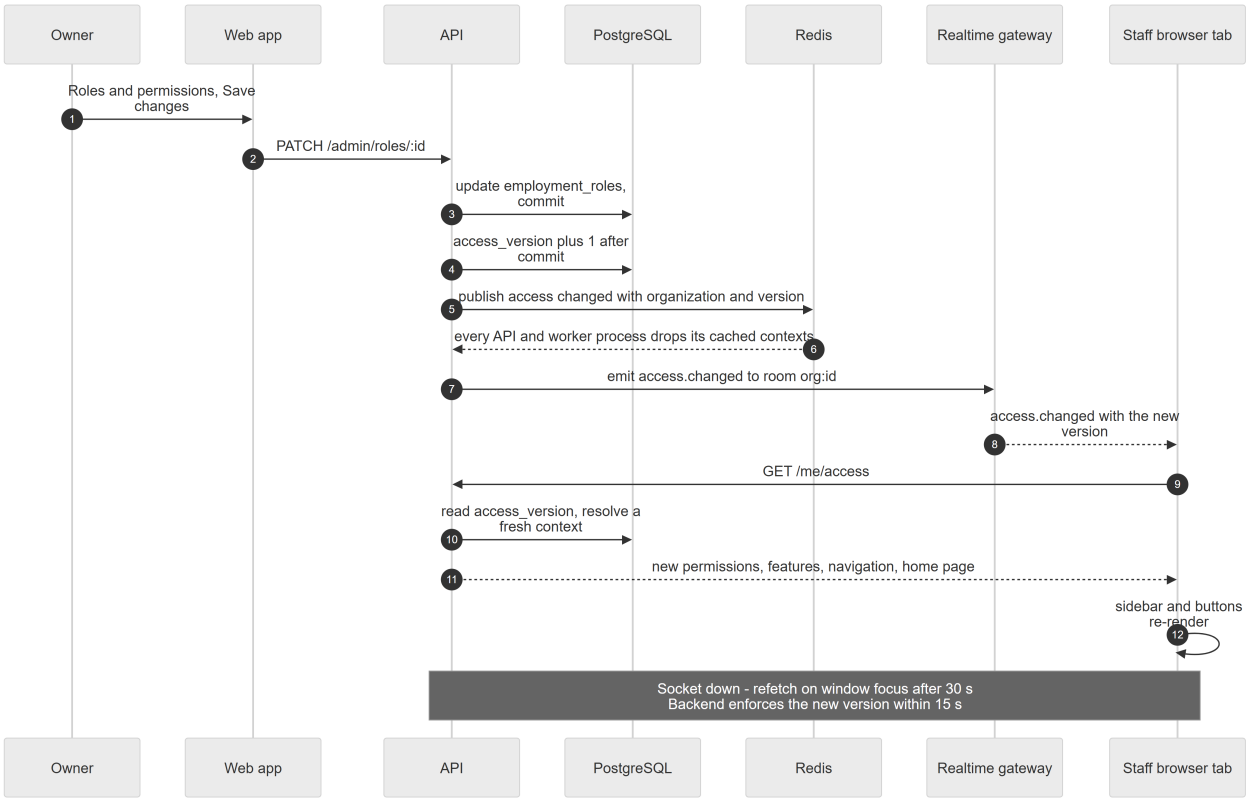


Figure 54.1 — From Save changes to a re-drawn sidebar.

The diagram has seven participants: Owner, Web app, API, PostgreSQL, Redis, Realtime gateway and Staff browser tab, and 12 messages plus one note. The Owner presses Save changes on the role (`PATCH /admin/roles/:id`). The API updates `employment_roles` and commits, then raises `access_version` by one after the commit. It publishes "access changed" with the organization and version to Redis, and every API and worker process drops its cached contexts. The API tells the realtime gateway to emit `access.changed` to the room for the organization, and the gateway sends it to the Staff browser tab with the new version. The tab calls `GET /me/access` ; the API reads the version, resolves a fresh context and returns new permissions, features, navigation

and home page, and the sidebar and buttons re-render. The note says that if the socket is down, the tab refetches on window focus once its data is 30 seconds old, and the backend still enforces the new version within 15 seconds.

Step 1 — The owner edits

Module: Roles and permissions, Features, Customization. **Status:** [Implemented]. Three kinds of edit start the chain: a role at **Roles & permissions**, a feature switch at **Features** or the entitlement page, and the clinic sidebar layout at **Navigation**. Granting a high-risk permission asks for a reason (Chapter 11, Chapter 10).

Step 2 — The version goes up

Module: Access control. **Status:** [Implemented]. `organizations.access_version` increases after any committed change to employment roles, assignments (role, location, active flag, portal access, login), organization feature flags or ownership. A database event subscriber does this automatically once per transaction, after the commit. Location feature overrides and nav layout changes bump it explicitly; saving the same nav layout again does not. The theme save has no bump in the code, so theme changes do not trigger this chain (Chapter 57).

Step 3 — Server caches drop

Module: Access control. **Status:** [Implemented]. The new version goes over Redis to every API and worker process, which drop cached contexts and feature sets. The resolver's cache is keyed on user, organization, location and access version, holds up to 10,000 entries and lasts 5 minutes; the version itself is trusted for 15 seconds without a bus message. If Redis is down, other processes catch up within 15 seconds. Because the key includes the version, a stale entry is never matched after a bump.

Step 4 — The browser is told

Module: Realtime. **Status:** [Implemented]. The gateway emits `access.changed` to the organization's room. The web app joins that room through the same socket used for the queue. A message whose version is not newer than the one the tab already has is ignored. After a reconnect the tab refetches in case messages were missed.

Step 5 — The tab refetches and redraws

Module: Access control (web app). **Status:** [Implemented]. The tab invalidates its access query, loads `GET /me/access` and redraws the sidebar, buttons and home page from `can(...)`. Route permission checks happen when the user navigates. If the room is refused or the socket is down, the fallback is refetch on window focus with a 30-second freshness window (Chapter 57).

Step 6 — Enforcement is already updated

Module: Access control (backend). **Status:** [Implemented]. The backend never trusts the screen. The access guard resolves access again on each request using the current version, so a removed permission is refused as soon as the version bump is visible to that process, even if the tab has not redrawn. A missing key means no access (fail-closed).

54.3 What can go wrong

- Redis is down: other server processes may take up to 15 seconds; tabs rely on window focus.
- Socket blocked or offline: the tab keeps old buttons until focus or the next reload, but the server still refuses actions.

- Raw SQL edits to roles do not bump the version unless done explicitly.
- A bump failure after commit is logged and the change itself stays saved, so caches may age out only on their own timers.
- Escalation protection: granting more than the editor holds is refused (`ESCALATION`).
- Super admin acting in the clinic is recorded as a platform action.

54.4 Data left behind

`organizations.access_version` , `employment_roles` (or `org_feature_flags` , `location_feature_overrides` , `nav_layouts`), `audit_logs` entries for the edit, and short-lived in-memory caches that hold no lasting data.

Chapter 55 — Workflow: From Purchase Order to Consumption

This chapter follows a box of gloves from a purchase order to stock, then to use on a patient and to a low-stock alert. It is written for the product owner. The honest summary is that the chain is only partly connected: the pieces exist in the backend, several have no screen, two steps carry a database risk that could not be tested, and the final alert reaches nobody. Each link in the chain carries its own label.

55.1 Who is involved

- **Admin**: the only system role with inventory write permissions (`inventory.po.manage` , `inventory.stock.adjust`). The Assistant role can only view.
- **Worker process**: runs the alert job.
- Features: `inventory.core` and `inventory.alerts` , both off by default.

55.2 The steps

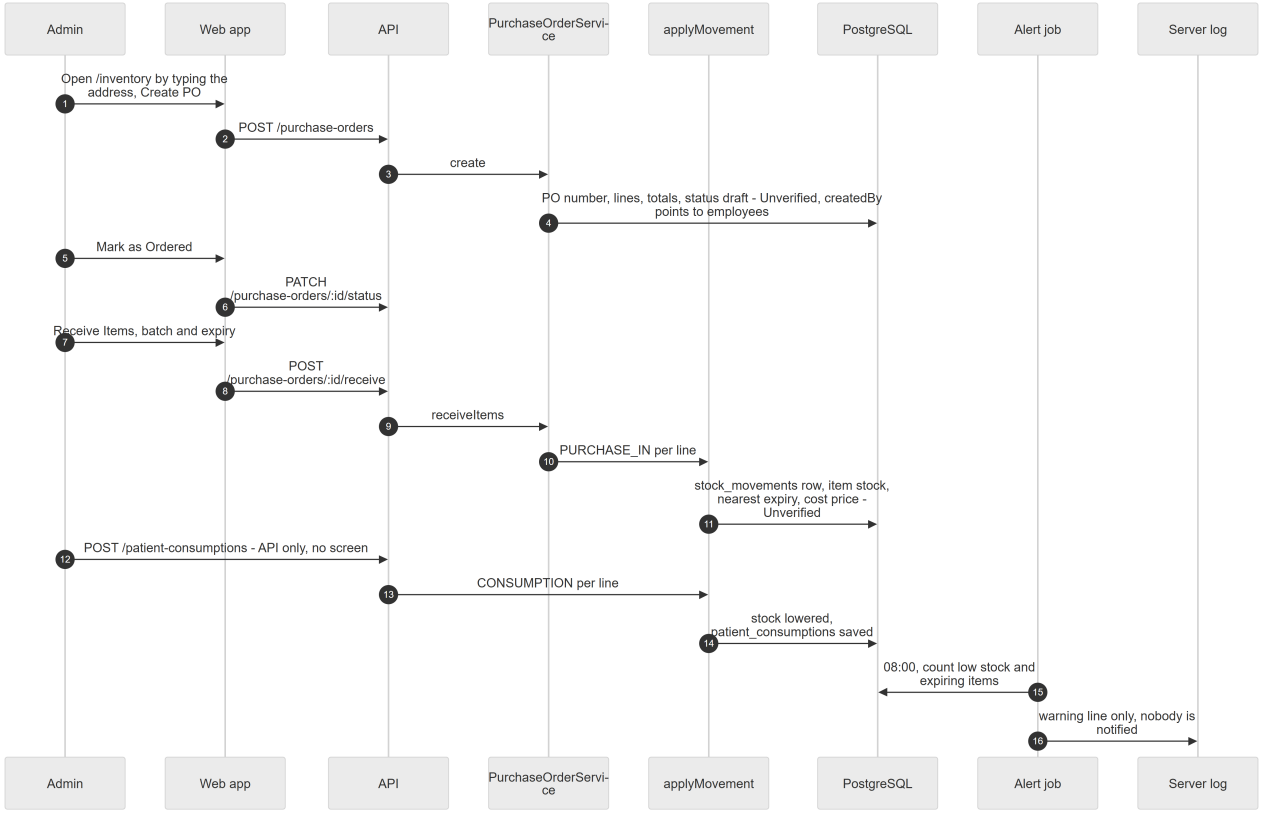


Figure 55.1 — The inventory chain and where it breaks.

The diagram has eight participants: Admin, Web app, API, PurchaseOrderService, applyMovement, PostgreSQL, Alert job and Server log, and 16 messages. The Admin opens `/inventory` by typing the address and creates a purchase order (`POST /purchase-orders`); the service stores the order number, lines, totals and status draft, with the Unverified note that the creator column points to employees. The Admin marks it ordered (`PATCH /purchase-orders/:id/status`), then receives items with batch and expiry (`POST /purchase-orders/:id/receive`). The service calls applyMovement with PURCHASE_IN per line, which writes a stock movement row and updates the item's

stock, nearest expiry and cost price, again marked Unverified. A call to `POST /patient-consumptions` has no screen; it lowers stock per line and saves the consumption. At 08:00 the alert job counts low and expiring items and writes only a warning line to the server log.

Link 1 — Reach the page

Module: Inventory. **Status:** [Partial]. The page `/inventory` works and needs `inventory.view`, but there is no sidebar entry and no screen links to it. Users must type the address. The feature `inventory.core` must be entitled and enabled first (Chapter 39).

Link 2 — Purchase order

Module: Inventory. **Status:** [Unverified]. Screens and logic exist: **Create PO** with supplier, lines, quantity, unit price and tax; the server numbers it like `PO-2026-00001` and computes totals; status goes Draft then Ordered (**Mark as Ordered**), and only a draft can be cancelled. The risk: creation stores the signed-in user's id in `purchase_orders.createdBy`, which has a foreign key to `employees`. A user id and an employee id are different records, so the insert is expected to fail. Reading the entity and the controller confirms that the user id is passed to that column; it was not run. PO numbers are also unique across the whole database but counted per organization, so a second organization's first order of a year can collide (deduced from code). Purchase orders are not linked to expenses or lab payables (Chapter 39).

Link 3 — Receive goods

Module: Inventory. **Status:** [Unverified]. **Receive Items** on an Ordered or Partially received PO takes quantities, batch and expiry per line; over-receiving is refused; the status becomes Partially received or Received. Each line adds a "purchase in" stock movement through the single `applyMovement` path, sets the item's cost price to the line price and moves the item's expiry date earlier if the batch expires sooner. It carries the same kind of foreign-key risk as the purchase order, through `stock_movements.performedBy`.

Link 4 — Stock movement and manual adjustment

Module: Inventory. **Status:** [Unverified]. Every quantity change is a row in `stock_movements` with previous and new stock; stock cannot go below zero. The Adjust stock dialog offers manual adjustment, wastage, return to supplier and transfers (notes required for wastage and manual). A transfer is recorded at one location only; nothing is added at the other. The item row is read without a row lock, so two simultaneous adjustments could start from the same number.

Link 5 — Patient consumption

Module: Inventory. **Status:** [Not implemented] in the web app. `POST /patient-consumptions` exists and prices each line at the item's current cost, then lowers stock in one transaction, but the component that would call it (`ConsumptionRecorder`) is placed on no screen, including the consultation workspace. Consumption templates (kits) can be created but nothing uses them. Consumed materials are not billed and do not show on the encounter (Chapter 39, Chapter 31).

Link 6 — Low-stock alert

Module: Inventory. **Status:** [Partial]. At 08:00 server time, for every location where both features are on, the job counts low-stock items and items expiring within 30 days and writes one warning line to the server log. No user is notified, no inbox entry is created and nothing appears on a screen; a code comment says notifications are planned (Chapter 44, Appendix E).

55.3 What can go wrong

- The first purchase order, receive or adjustment may fail with a database foreign-key error, which would stop the whole chain [Unverified].
- The page is not discoverable without a sidebar entry.
- "Insufficient stock" when an outgoing movement would go below zero.
- "Cannot receive more than ordered".
- Alerts are visible only to whoever reads server logs.
- Stock movement history is the only audit trail; no separate log exists.
- With `inventory.core` off, nothing in the module is permitted.

55.4 Data left behind

`purchase_orders`, `purchase_order_items`, `stock_movements`, `inventory_items` (stock, expiry, cost), `patient_consumptions` and `patient_consumption_items` (API only), and server log lines from the alert job. No billing, ledger or notification rows are written by any step.